

REPORT

과제명: PCA를 이용한 얼굴 인식 시스템 구현



교과목

컴퓨터 비전

학 과

응용소프트웨어공학과

학 번

20213032

이 름

권영훈

제출일자

2025.12.04



동의대학교
DONG-EUI UNIVERSITY

소스코드

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

# -----
# [설정] 상수 정의
# -----
IMG_H = 25 # 세로 크기
IMG_W = 20 # 가로 크기
N = IMG_H * IMG_W # 벡터 차원 (500)

# 파일 경로
TRAIN_DIR = './face_img_25_all/train/'
TEST_DIR = './face_img_25_all/test/'

def load_image_as_vector(path):
    """
    이미지를 읽어서 그레이스케일 변환, 25x20 리사이즈,
    1 차원 벡터(500x1)로 변환 후 float32 형으로 반환 [cite: 737, 738, 739, 741]
    """
    img = cv2.imread(path)
    if img is None:
        return None

    # 1. 명암도(Grayscale) 변환
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # 2. 크기 변환 (20x25)
    resized = cv2.resize(gray, (IMG_W, IMG_H))

    # 3. 1 차원 벡터 변환 및 float32 형변환
    vector = np.array(resized, dtype=np.float32).flatten().reshape(N, 1)

    return vector

def load_dataset(directory):
    """
    폴더 내의 모든 이미지를 읽어 행렬 X 구성
    반환: X (500 x P 크기), 파일명 리스트
    """
    file_list = sorted(os.listdir(directory)) # 파일 순서 정렬
    valid_files = []
    vectors = []

    for f in file_list:
        path = os.path.join(directory, f)
        vec = load_image_as_vector(path)
        if vec is not None:
            vectors.append(vec)
            valid_files.append(f)

    # (500 x P) 행렬로 병합
```

```

    if len(vectors) > 0:
        X = np.hstack(vectors)
    else:
        X = np.empty((N, 0))

    return X, valid_files

# -----
# [학습 단계]
# -----
print("1. 학습 데이터를 로딩 중입니다...")
X, train_filenames = load_dataset(TRAIN_DIR)
P = X.shape[1] # 학습 영상 개수 (308 개 예상)
print(f"    학습 데이터 크기(X): {X.shape}") # (500, 308)

# (1) 평균 영상(m) 구하기
# axis=1 은 각 행(pixel 위치)별 평균을 의미
m = np.mean(X, axis=1, keepdims=True)

# 평균 영상 저장 (원본 25x20 -> 150x150 확대)
# 1. 벡터를 25x20 이미지로 복원
mean_face_small = m.reshape(IMG_H, IMG_W).astype(np.uint8)

# 2. 보고서용으로 150x150 크기로 확대 (cv2.INTER_LINEAR 사용)
mean_face_enlarged = cv2.resize(mean_face_small, (150, 150),
interpolation=cv2.INTER_LINEAR)

# 3. 파일 저장
cv2.imwrite('mean_face.jpg', mean_face_enlarged)
print("    평균 영상 저장 완료 (mean_face.jpg, 150x150 크기)")

# (2) 차 영상 행렬(X_bar) 구하기: 각 영상에서 평균 빼기
X_bar = X - m

# (3) 공분산 행렬(Covariance Matrix) 구하기
# 이론 수식: Omega = X_bar * X_bar.T
# 결과 크기: (500 x 308) * (308 x 500) = (500 x 500)
Omega = np.dot(X_bar, X_bar.T)

# (4) 고유값(Eigenvalues)과 고유벡터(Eigenvectors) 구하기
# np.linalg.eig 사용 (PCA 함수 아님, 선형대수 기본 연산)
eig_vals, eig_vecs = np.linalg.eig(Omega)

# (5) 고유값 정렬 (내림차순) 및 주요 성분 선택
sort_indices = np.argsort(eig_vals)[::-1]
sorted_eig_vals = eig_vals[sort_indices]
sorted_eig_vecs = eig_vecs[:, sort_indices]

# [과제 제출용] 고유값 분포 그래프 저장
plt.figure()
plt.plot(sorted_eig_vals[:50]) # 상위 50 개 정도만 확인
plt.title("Eigenvalues Distribution")
plt.xlabel("Index")
plt.ylabel("Value")
plt.grid(True)

```

```

plt.savefig('eigenvalues_plot.png')
print("    고유값 분포 그래프 저장 완료 (eigenvalues_plot.png)")

# (6) 변환 행렬 V 생성 (상위 K 개 선택)
# 예: 상위 30 개 주성분 사용 (실험을 통해 K 값 조절 가능)
K = 30
V = sorted_eig_vecs[:, :K] # (500 x K)

# (7) 학습 영상 투영 (특징 벡터 추출)
# Y_train = V.T * X_bar
# 결과 크기: (K x 500) * (500 x 308) = (K x 308)
Y_train = np.dot(V.T, X_bar)

print("    학습 완료. 모델이 준비되었습니다.")

# -----
# [테스트 단계]
# -----
print("\n2. 테스트를 시작합니다. (종료하려면 'Q' 또는 'q' 입력)")

while True:
    # 1. 사용자 입력 (반복 루프 필수)
    user_input = input("\n-> 파일 번호 입력 (예: 25) or 종료(Q): ")

    if user_input.lower() == 'q':
        print("프로그램을 종료합니다.")
        break

    # 입력 번호 포매팅
    try:
        filename = f"test{int(user_input):03d}.jpg"
        test_path = os.path.join(TEST_DIR, filename)
    except ValueError:
        print("잘못된 입력입니다. 숫자를 입력하세요.")
        continue

    # 2. 테스트 영상 로드 및 전처리
    input_vec = load_image_as_vector(test_path)

    if input_vec is None:
        print(f"파일을 찾을 수 없습니다: {test_path}")
        continue

    # 3. 테스트 영상 투영
    # (1) 평균 영상 빼기: y_bar = y - m
    input_vec_bar = input_vec - m

    # (2) 투영: y_proj = V.T * y_bar
    input_proj = np.dot(V.T, input_vec_bar) # (K x 1)

    # 4. 거리 계산 (유클리디안 거리) 및 최소값 탐색
    # 학습된 모든 특징 벡터(Y_train)와 입력 특징 벡터(input_proj) 간 거리 계산
    min_dist = float('inf')
    min_idx = -1

```

```

for i in range(P):
    # 학습 영상 i의 특징 벡터
    train_vec = Y_train[:, i].reshape(-1, 1)

    # 유클리디안 거리 계산: sqrt(sum((a-b)^2))
    dist = np.linalg.norm(input_proj - train_vec)

    if dist < min_dist:
        min_dist = dist
        min_idx = i

# 5. 결과 출력
# 찾은 학습 영상 파일명
found_filename = train_filenames[min_idx]
print(f"  입력: {filename} -> 인식 결과: {found_filename} (거리: {min_dist:.2f})")

# (1) 원본 이미지 읽기 (화면 출력용)
img_input = cv2.imread(test_path)
img_found = cv2.imread(os.path.join(TRAIN_DIR, found_filename))

# (2) 150x150 크기로 확대
if img_input is not None and img_found is not None:
    disp_input = cv2.resize(img_input, (150, 150))
    disp_found = cv2.resize(img_found, (150, 150))

    # (3) 두 영상을 나란히 붙이기 (Numpy hstack)
    display_img = np.hstack((disp_input, disp_found))

    # (4) 윈도우에 출력
    window_name = f"Input: {user_input} / Found: {found_filename}"
    cv2.imshow("Face Recognition Result", display_img)
    cv2.setWindowTitle("Face Recognition Result", window_name)

    print("  결과 창을 닫거나 아무 키나 누르면 다음으로 넘어갑니다.")
    cv2.waitKey(0) # 키 입력 대기
    cv2.destroyAllWindows()
else:
    print("  이미지 출력 오류")

```

소스코드 설명

1. 데이터 전처리 및 벡터화

```
def load_image_as_vector(path):  
    # 영상을 읽어옵니다.  
    img = cv2.imread(path)  
    if img is None:  
        return None  
  
    # 1. 명암도(Grayscale) 변환 [이론: 컬러 영상을 명암도 영상으로 변환]  
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  
  
    # 2. 크기 변환 (20x25) [이론: 모든 영상의 크기는 동일해야 함 (H, W)=(25, 20)]  
    resized = cv2.resize(gray, (IMG_W, IMG_H))  
  
    # 3. 1차원 벡터 변환 (500x1) [이론: 영상을 1차원 벡터(N*1)로 표현]  
    # float32로 변환하여 연산 정밀도 확보  
    vector = np.array(resized, dtype=np.float32).flatten().reshape(N, 1)  
  
    return vector
```

구현 설명

load_image_as_vector 함수에서 컬러 영상을 명암도 영상으로 변환하고, 25 * 20 크기로 리사이즈했습니다. 이후 flatten()을 사용하여 500 * 1 크기의 1차원 벡터로 변환하였습니다

2. 평균 영상 및 차 영상 계산

```
# (1) 평균 영상(m) 구하기: axis=1은 각 행(pixel)별 평균  
m = np.mean(X, axis=1, keepdims=True)  
  
# (2) 차 영상 행렬(X_bar) 구하기: 각 영상에서 평균 빼기  
X_bar = X - m
```

구현 설명

학습 데이터 행렬 X (500 * 308)의 각 행(row)에 대한 평균을 구하여 평균 벡터 m을 계산했습니다. 그 후, 원본 학습 데이터 X에서 평균 m을 빼주어 데이터의 중심을 0으로 맞추는 차 영상 행렬X를 구했습니다

3. 공분산 행렬 계산 및 고유값 분해 (PCA 수행)

```
# (3) 공분산 행렬(Omega) 직접 계산
# 이론 수식: Omega = X_bar * X_bar.T
Omega = np.dot(X_bar, X_bar.T)

# (4) 고유값과 고유벡터 추출 (선형대수 기본 함수 사용)
eig_vals, eig_vecs = np.linalg.eig(Omega)

# (5) 고유값 크기순 정렬 (내림차순)
sort_indices = np.argsort(eig_vals)[::-1]
sorted_eig_vecs = eig_vecs[:, sort_indices]
```

구현 설명

PCA 라이브러리를 쓰지 않고, np.dot 함수를 이용해 직접 계산하여 500 * 500 크기의 공분산 행렬을 만들었습니다. 이 행렬에 대해 np.linalg.eig 함수를 적용하여 고유값과 고유벡터를 추출했습니다. 구해진 고유값들은 정보량이 많은 순서(내림차순)로 정렬하여 이후 주성분 선택에 사용했습니다.

4. 특징 벡터 추출 및 테스트 (유클리디안 거리)

```
# (6) 변환 행렬 V 생성 (상위 30 개 주성분 선택)
K = 30
V = sorted_eig_vecs[:, :K]

# (7) 학습 영상 투영하여 특징 벡터 구하기
Y_train = np.dot(V.T, X_bar)

# --- 테스트 루프 중 거리 계산 부분 ---
# 테스트 영상 투영
input_vec_bar = input_vec - m
input_proj = np.dot(V.T, input_vec_bar)

# 최소 거리 탐색
min_dist = float('inf')
min_idx = -1

for i in range(P):
    train_vec = Y_train[:, i].reshape(-1, 1)
    # 유클리디안 거리 계산: sqrt(sum((a-b)^2))
    dist = np.linalg.norm(input_proj - train_vec)

    if dist < min_dist:
        min_dist = dist
        min_idx = i
```

구현 설명

정렬된 고유벡터 중 상위 30개(K = 30)를 주성분으로 선택하여 변환 행렬 V를 생성했습니다. 이를 통해 고차원(500)의 영상 데이터를 저차원(30)의 특징 벡터로 투영했습니다. 테스트 단계에서는 입력 영상의 특징 벡터와 학습 영상들의 특징 벡터 사이의 유클리디안 거리를 계산하였으며, 이 거리가 최소가 되는 학습 영상을 찾아 최종 인식 결과로 선정했습니다.

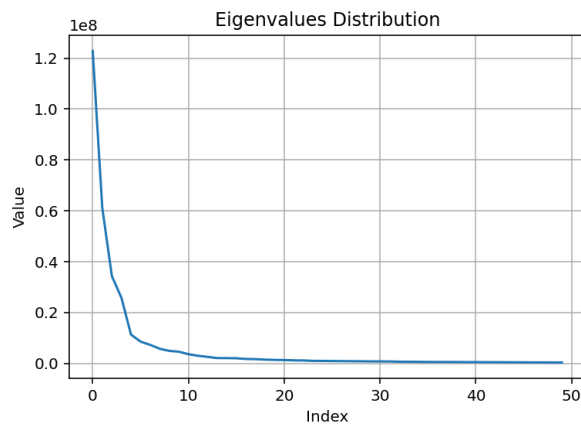
결과 출력

평균 영상 (Mean_Face)



학습 데이터 308개의 평균을 시각화한 영상입니다

고유값 분포 그래프 (eigenvalues_plot)



고유값을 크기순으로 정렬한 분포입니다.
앞부분의 주성분이 데이터의 대부분을 설명함을 알 수 있습니다.

인식 결과 (테스트 화면 캡처)



-> 파일 번호 입력 (예: 25) or 종료(Q): 47
입력: test047.jpg -> 인식 결과: train160.jpg (거리: 63.22)
결과 창을 닫거나 아무 키나 누르면 다음으로 넘어갑니다.

본인 이미지 사용 (입력: test047.jpg -> 결과: train160.jpg)